W3 — React ve JSX | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W3
Kapsam: Bileşenler, JSX, props, state, yaşam döngüsü, events, koşul/listeler, hooks, veri paylaşımı, stiller, formlar ve React Router.
DİKKAT: W3 deposundaki örneklerin büyük kısmı **web React** içindir (`div`, `input`, CSS, React Router). Bunları Expo Snack'e aynen yapıştırmak doğru değildir. Mobil karşılıkları W4/W6 dosyalarında verildi.

1. React nedir? Bileşen mantığı

React, kullanıcı arayüzlerini **bileşen** (component) adı verilen tekrar kullanılabilir parçalardan oluşturan bir JavaScript kütüphanesidir. UI, state ve props değişimlerine göre yeniden hesaplanır. Bileşen adları **büyük harfle** başlar (`MerhabaKart`). Modern projelerde fonksiyon bileşenleri ve Hook'lar en yaygın yaklaşımdır.

```
function Karsilama({ ad }) {
  return <h2>Merhaba {ad}!</h2>;
}
export default function App() {
  return <Karsilama ad="Ünal" />;
}
```

Kural: `<Karsilama />` bir React bileşenidir; `<h2>` bir HTML elementidir. `function Karsilama()` kodu JSX döndürür. `export default` başka dosyalardan bu bileşeni import etmeyi sağlar.

Fonksiyon ve sınıf bileşeni

```
// Fonksiyon bileşeni
function Baslik({ metin }) { return <h1>{metin}</h1>; }

// Eski stil: class bileşeni
class Selam extends React.Component {
  render() { return <h1>Selam {this.props.ad}</h1>; }
}
```

Sınıf bileşenleri `this.state`, `setState`, `componentDidMount` gibi API'ler kullanır. Modern fonksiyon bileşenleri `useState`, `useEffect` kullanır.

2. JSX kuralları

JSX, JavaScript içinde HTML'ye benzeyen UI yazma biçimidir. JSX ifadesinde JavaScript hesaplamaları `{ ... }` içine yazılır. Önemli kurallar:

1. Tek bir kök element döndür: `<div>...</div>` veya `<>...</>`.
2. HTML tarafında `class` yerine `className`, `for` yerine `htmlFor` kullan.
3. Açık/kapalı etiketleri eşleştir; boş etiketleri `<img />` şeklinde yaz.
4. `onClick` olayına **fonksiyon** ver, `onClick={tikla}`; `onClick={tikla()}` yazarsan render sırasında çağırırsın.

```
const ad = 'Ünal';
const puan = 70;
const ekran = (
  <div className="kart">
    <h3>{ad.toUpperCase()}</h3>
    <p>Sonuç: {puan >= 50 ? 'Geçti' : 'Kaldı'}</p>
    <img src="/logo.png" alt="Logo" />
  </div>
);
```

Mobil ayrımı: React Native'de `<div>`, `<h3>`, `<p>`, `<img>` **yoktur**; genellikle `<View>`, `<Text>`, `<Image>` kullanılır. `className` yerine çoğunlukla `style` kullanılır.

3. Props (bileşenler arası veri aktarma)

Props = üst bileşenden alt bileşene gönderilen ve alt bileşenin değiştirmemesi gereken girdiler. Props, fonksiyonun parametresi gibidir. `children`, etiketin arasındaki içeriği taşır.

```
function Kart({ baslik = 'Başlıksız', children }) {
  return (
    <section>
      <h3>{baslik}</h3>
      <div>{children}</div>
    </section>
  );
}
function App() {
  return <Kart baslik="Sınav"><p>Çalışmaya devam!</p></Kart>;
}
```

`baslik = 'Başlıksız'` varsayılan parametre. `children` içine gönderilen JSX. Alt bileşen `props.baslik = ...` yapmamalı. Değişiklik gerekiyorsa parent state'ini callback ile güncelle.

4. State ve `useState`

State = bileşenin değişebilen belleği. Değiştiğinde React yeni render yapar. `useState(başlangıç)` → `[değer, güncellemeFonksiyonu]`.

```
import { useState } from 'react';

export default function Sayac() {
  const [sayac, setSayac] = useState(0);
  return (
    <div>
      <p>Sayac: {sayac}</p>
      <button onClick={() => setSayac(s => s + 1)}>+1</button>
    </div>
  );
}
```

Neden `setSayac(s => s + 1)`? Güncelleme eski değere bağlıysa fonksiyonel biçim bekleyen güncellemelerle güvenilir çalışır.

```
// YANLIŞ: state doğrudan mutasyona uğratılır.
// kullanicilar.push(yeniKullanici);
// setKullanicilar(kullanicilar);

// DOĞRU: yeni dizi üret.
setKullanicilar(eski => [...eski, yeniKullanici]);
setKullanicilar(eski => eski.filter(k => k.id !== silinecekId));
```

Props	State
Parent'tan gelir	Bileşenin kendisi yönetir
Alt bileşen açısından salt okunur	Setter ile değiştirilir
Örn. `<Kart isim="Ünal" />`	Örn. `[isim, setIsim] = useState('Ünal')`

5. Etkinlik yönetimi (Events)

```
function Dugme() {
  function tiklandi() { alert('Butona basıldı'); }
  return <button onClick={tiklandi}>Bas</button>;
}
```

React web: `onClick`, `onChange={e => ...}`, `e.target.value`, `onSubmit`, `e.preventDefault()`. **React Native:** tıklama için `onPress`, metin girişi için `onChangeText` kullanılır; tarayıcı `event.target.value` kalıbı doğrudan Expo'da aynı değildir.

6. Koşullu gösterim ve liste

```
function Durum({ girisYapti }) {
  if (!girisYapti) return <p>Lütfen giriş yap</p>;
  return <p>Hoş geldin!</p>;
}

function Rozet({ puan }) {
  return <strong>{puan >= 50 ? 'Geçti' : 'Kaldı'}</strong>;
}

function Liste({ kisiler }) {
  return <ul>{kisiler.map(k => <li key={k.id}>{k.ad}</li>)}</ul>;
}
```

- `&&` ile gösterim: `{hazir && <p>Hazır</p>}`.
- **Tuzak:** `{sayi && <p>Var</p>}` ifadesi `sayi = 0` ise ekranda `0` oluşturabilir; `{sayi > 0 && ...}` daha güvenlidir.
- `key` benzersiz ve **kararlı** olmalıdır: verinin `id` alanı idealdir. Değişen listelerde dizin (`index`) key olarak sorun çıkarabilir.
- React'te `map` UI elemanları üretir; `filter` görünmesini istemediklerini çıkarır.

7. `useEffect`: yan etki ve yaşam döngüsü

Yan etki (side effect): Ağ isteği, abonelik, zamanlayıcı, harici API gibi render dışı işler. `useEffect` bu işler için kullanılır.

```
import { useEffect, useState } from 'react';
function Saat() {
  const [saniye, setSaniye] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setSaniye(s => s + 1), 1000);
    return () => clearInterval(id); // temizleme
  }, []);
  return <p>{saniye} saniye</p>;
}
```

Biçim	Genel davranış
`useEffect(fn)`	Her commit sonrası çalışabilir
`useEffect(fn, [])`	Bileşen mount olduğunda; development Strict Mode'da ek test çalışmaları olabilir
`useEffect(fn, [id])`	Mount ve `id` değiştiğinde çalışır
`return () => ...`	Temizleme: bağımlılık değişmeden önce/unmount sırasında

Önemli: Hook'u `if` içinde veya döngüde çağırma. Hook çağrı sırası her render'da aynı kalmalıdır. `useEffect(async () => ...)` yerine effect içinde async fonksiyon tanımlayıp çağırmak daha uygundur.

8. Diğer önemli Hook'lar

Hook	Ne işe yarar?	Kısa örnek
`useState`	Yerel değişen değer	`const [x,setX] = useState(0)`
`useEffect`	Yan etki / abonelik	`useEffect(() => {...}, [])`
`useRef`	Render tetiklemeden kalıcı referans	`const ref = useRef(null)`
`useContext`	Uzak bileşene ortak veri	`useContext(ThemeContext)`
`useReducer`	Karmaşık state geçişleri	`useReducer(reducer, initial)`
`useMemo`	Hesaplanan değeri önbellekleme	`useMemo(() => hesapla(x), [x])`
`useCallback`	Fonksiyon referansını koruma	`useCallback(() => kaydet(id), [id])`

`useMemo` ve `useCallback` çoğu küçük projede şart değildir; performans sorunu varsa anlamlıdır. `useRef.current` güncellenmesi tek başına render tetiklemez.

```
const [adet, setAdet] = useState(1);
const ikiKat = useMemo(() => adet * 2, [adet]);
const arttir = useCallback(() => setAdet(x => x + 1), []);
```

9. Bileşenler arasında veri paylaşımı

En sık çözüm **lifting state up**: Ortak veriyi iki alt bileşenin **en yakın ortak üst bileşeninde** tut ve aşağı props/callback gönder.

```
function Goster({ deger }) { return <p>{deger}</p>; }
function Arttir({ onArttir }) {
  return <button onClick={onArttir}>Artır</button>;
}
function App() {
  const [n, setN] = useState(0);
  return <><Goster deger={n} /><Arttir onArttir={() => setN(x => x + 1)} /></>;
}
```

Prop drilling: Çok sayıda katmandan props geçirme. Çok yaygın ve ortak veriler için Context API, büyük ölçekli ihtiyaçlar için ek state kütüphaneleri kullanılabilir.

10. Web React stilleri ve CSS

```
const kartStili = { backgroundColor: '#eef', padding: 16 };
function Kart() {
  return <div className="kart" style={kartStili}>İçerik</div>;
}
```

React web'de haricî `.css` dosyası, CSS Modules, CSS-in-JS, Tailwind kullanılabilir. `style` **JavaScript nesnesi** alır; `background-color` yerine `backgroundColor` yazılır. React Native'de ayrıca `StyleSheet.create()` vardır ve CSS dosyası React Native çekirdek bileşenlerine doğrudan uygulanmaz.

11. Formlar ve kontrollü bileşen

Kontrollü formda `input` değerinin kaynağı React state'idir.

```
import { useState } from 'react';
function AdFormu() {
  const [ad, setAd] = useState('');
  const [hata, setHata] = useState('');

  function gonder(e) {
    e.preventDefault();
    setHata(ad.trim() ? '' : 'Ad boş bırakılamaz');
    if (ad.trim()) alert('Merhaba ' + ad);
  }
  return (
    <form onSubmit={gonder}>
      <input value={ad} onChange={e => setAd(e.target.value)} />
      {hata && <p>{hata}</p>}
      <button type="submit">Gönder</button>
    </form>
  );
}
```

Kontrollü: `value={state}` ve `onChange` birlikte. **Kontrolsüz:** Değer DOM üzerinde tutulur (`ref`). React Native'de benzer kontrollü kalıp `<TextInput value={ad} onChangeText={setAd} />` biçimindedir.

12. Web yönlendirmesi: React Router

React Router, URL değiştiğinde başka sayfa bileşeni göstermeye yarar. **W3 konusu web React içindir**; Expo uygulamasında genellikle Expo Router veya React Navigation kullanılır.

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
function AnaSayfa() { return <h2>Ana sayfa</h2>; }
function Hakkinda() { return <h2>Hakkında</h2>; }
function App() {
  return (
    <BrowserRouter>
      <nav><Link to="/">Ana</Link> | <Link to="/hakkinda">Hakkında</Link></nav>
      <Routes>
        <Route path="/" element={<AnaSayfa />} />
        <Route path="/hakkinda" element={<Hakkinda />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Not: Bu örnek W3'teki React Router v6 tarzını gösterir. Paket sürümüne göre bazı React Router API'leri farklı olabilir.

13. Mini proje — Web React görev listesi (tam örnek)

Bu kod **React web** projesinde `App.jsx` içine uygundur. **Expo Snack'e doğrudan yapıştırma:** `<div>`, `<input>`, `<button>` gibi web etiketleri içerir.

```
import { useState } from 'react';
export default function App() {
  const [yeni, setYeni] = useState('');
  const [gorevler, setGorevler] = useState([
    { id: 1, ad: 'JS tekrar et' },
  ]);
  function ekle() {
    if (!yeni.trim()) return;
    setGorevler(eski => [...eski, { id: Date.now(), ad: yeni.trim() }]);
    setYeni('');
  }
  return (
    <div>
      <h1>Görev Listem</h1>
      <input value={yeni} onChange={e => setYeni(e.target.value)} />
      <button onClick={ekle}>Ekle</button>
      <ul>
        {gorevler.map(g => (
          <li key={g.id}>
            {g.ad}
            <button onClick={() => setGorevler(eski => eski.filter(x => x.id !== g.id))}>
              Sil
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

14. W3 React web → Expo React Native hızlı dönüşüm

W3 (Web React)	W4/W6 (React Native)
`<div>`	`<View>`
`<p>`, `<span>`, `<h1>`	`<Text>` + `style`
`<button onClick={fn}>`	`<Button title="Bas" onPress={fn} />`
`<input onChange={e => ...}>`	`<TextInput onChangeText={...} />`
`<img src="..." />`	`<Image source={{ uri: '...' }} style={{width:...,height:...}} />`
CSS `.kart { ... }`	`StyleSheet.create({ kart: { ... } })`
`react-router-dom`	Expo Router / React Navigation
`useState`, `useEffect`	Aynı Hook'lar kullanılır

15. Sınav soruları ve kısa cevaplar

S1. `props` ve `state` farkı nedir? **Cevap:** Props dışarıdan gelen salt-okunur girdiler; state bileşenin güncellenen yerel verisidir.

S2. `onClick={arttir()}` neden sorunlu? **Cevap:** Fonksiyonu render sırasında çağırır; olay işleyicisi olarak fonksiyon referansı verilmeli (`onClick={arttir}`).

S3. `useEffect(..., [])` hangi duruma karşılık gelir? **Cevap:** Mount ile ilişkili effect; temizleme unmount'ta, geliştirme Strict Mode özel davranabilir.

S4. `key` neden gerekli? **Cevap:** React'in listedeki öğelerin kimliğini koruyup güncellemeleri doğru ilişkilendirmesi için.

S5. `setListe([...liste, yeni])` ile `liste.push(yeni)` farkı? **Cevap:** İlki yeni dizi oluşturur ve React state güncellemesine uygundur; ikincisi mevcut diziyi değiştirir.

S6. `useRef` güncellemesi her zaman yeniden render yapar mı? **Cevap:** Hayır.

S7. `children` nedir? **Cevap:** Bileşen etiketlerinin arasındaki JSX içeriğidir.

S8. Web React kodunda `<button>` yerine Expo'da hangi bileşen? **Cevap:** `Button`, `Pressable` veya `TouchableOpacity`.

16. Hızlı başvuru

Sorun	Kalıp
Bileşen oluştur	`function Kart({ad}) { return <p>{ad}</p> }`
State oluştur	`const [x, setX] = useState(0);`
Eski state'e göre artır	`setX(v => v + 1);`
Koşullu render	`{hazir ? <A /> : <B />}`
Liste render	`{liste.map(x => <li key={x.id}>{x.ad}</li>)}`
Effect temizliği	`useEffect(() => { const id=...; return () => clearInterval(id); }, [])`
Parent'tan callback	`<Buton onArttir={() => setX(v => v+1)} />`

Kaynak dosyalar: W3/README.md, W3 bileşen ve JSX notları.
Bu dosya, kaynak depoya dayanan Türkçe sınav rehberidir; web ve mobil kodların farkları özellikle vurgulanmıştır.